

Hedera Hashgraph as a Scalable Distributed Ledger: Addressing Blockchain Scalability and Consensus Limitations (2026)

Mary Queenvine Muthoni, 158716, Emmanuel Keter, 134976, and Louis Karanja, 169275

Abstract—Blockchain and distributed ledger technologies (DLTs) have transformed decentralized systems but face important challenges in scalability, energy use, and transaction speed. These issues mainly arise from sequential block-based consensus methods like Proof-of-Work (PoW). This paper looks at Hedera Hashgraph as a next-generation DLT that addresses these challenges through a Directed Acyclic Graph (DAG) structure and an Asynchronous Byzantine Fault Tolerant (aBFT) consensus protocol. By using a gossip protocol and virtual voting, Hedera allows for fast and low-latency transaction processing without needing to shard or use off-chain solutions. This study examines the shift from traditional blockchains to DAG-based ledgers, showing how Hedera achieves reliable, leaderless consensus in real-world asynchronous contexts. The findings suggest that Hedera provides a strong, secure, and energy-efficient alternative for modern distributed applications, ensuring reliable finality where traditional probabilistic models fall short.

Index Terms—Distributed Systems, Scalability, aBFT, Hedera Hashgraph, DAG, Consensus, Fault Tolerance

I. INTRODUCTION

Distributed ledger technologies (DLTs) have fundamentally changed trust, decentralization, and data integrity in today's computing infrastructure (Bano et al., 2019). By creating shared, cryptographically verified data stores that are replicated across distrustful nodes, distributed ledgers remove single points of failure and the vulnerabilities found in centralized systems (Zheng et al., 2017). The evolution of distributed systems has moved from early linear blockchains to Directed Acyclic Graph (DAG) structures, reflecting a continuous effort to improve performance without compromising security (Wang et al., 2020). Despite their promise, traditional linear blockchains still face significant scalability issues. Global block ordering, linear sequencing, and high-overhead consensus mechanisms limit transaction speed while increasing latency and costs (Vukolić, 2015).

To thoroughly evaluate how modern distributed architectures overcome these challenges, this paper analyzes Hedera Hashgraph's design and functions. Hedera marks a crucial shift from traditional block-based systems. By substituting a sequential chain of blocks with a graph-structured DAG and employing an aBFT consensus protocol, Hedera shows how parallel transaction processing and message-efficient voting mechanisms can improve

performance at scale (Baird, 2016). This study looks at the key structural properties, network protocols, and mathematical guarantees that set Hedera Hashgraph apart from older decentralized platforms. It provides an assessment within the broader context of modern distributed systems design.

II. LITERATURE REVIEW

The development of distributed ledger architectures revolves around the ongoing effort to solve the scalability trilemma. This concept states that a decentralized system can only effectively meet two of three main properties: security, decentralization, and scalability (Buterin, 2017). In older platforms, performance issues stem from the connection between data storage and consensus sequences.

Traditional Proof-of-Work (PoW) systems, like Bitcoin, achieve consensus through competition (Nakamoto, 2008). Nodes compete to solve complex cryptographic puzzles to gain the right to add a localized group of transactions (a block) to a single, widely shared chain. This design creates an adversarial bottleneck. To prevent damaging chain splits (forks), the network must limit block generation, which leads to low throughput and long probabilistic finality times (Vukolić, 2015). With probabilistic finality, a transaction is never guaranteed; its security grows exponentially as more blocks are added, making it sensitive to network latency. To address the significant energy use and transaction limits of PoW, alternative models have introduced Proof-of-Stake (PoS) and leader-based systems (King & Nadal, 2012). Although PoS replaces competition with economic stake weighting, many of these models still rely on a designated leader or rotating committee to suggest the next state update.

Leader-based distributed systems face unique risks, such as targeted Distributed Denial of Service (DDoS) attacks on the active leader and extra overhead from view-change protocols when a leader fails or behaves maliciously (Castro & Liskov, 1999). Moreover, optimization methods like network sharding, which divides states among semi-autonomous groups of nodes, and Layer-2 off-chain networks add complexity, synchronization delays, and additional security issues (Wang et al., 2020).

In contrast, recent research on distributed systems has shifted towards leaderless, graph-based structures. DAGs separate transaction submission from global block creation, allowing nodes to make and share state updates asynchronously without waiting for a single coordinator (Baird, 2016).

Hedera Hashgraph fills these specific needs by using an aBFT protocol that provides reliable finality without

traditional blocks, heavy computational validation, or a local leader, establishing a new standard for fault tolerance in open, asynchronous networks (Baird et al., 2019).

III. METHODOLOGY

This research examines the structural mechanics and performance features of Hedera Hashgraph by analyzing its architectural layers and message propagation. The assessment focuses on overcoming the linear block bottleneck by evaluating data flow in two main layers: the structural graph generation layer (the Hashgraph DAG) and the mathematical agreement layer (the aBFT consensus algorithm) (Baird, 2016).

A. Network Topology and Graph Generation

The network consists of a group of independent validator nodes operating in an asynchronous environment where message delays are bounded but not fixed. Instead of a single sequence of blocks, the ledger state is held in a cryptographically verified DAG (Baird, 2016). Each event node in the hashgraph is a data structure containing:

- A payload of zero or more transactions.

- A timestamp marking the local creation time.

- Two parent cryptographic hashes: the self-parent hash (pointing to the producing node's previous event) and the other-parent hash (citing the latest event received from another node via gossip).

This dual-parent setup ensures that the graph reflects the complete history of network communication, creating an immutable, distributed record of how information flows through the system (Baird et al., 2019).

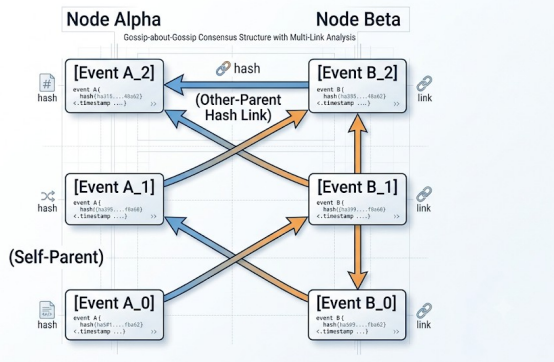


figure1: Network Topology

B. Protocol Foundations and Algorithmic Assumptions

The core protocol functions without relying on synchronization assumptions or global network clocks. The mathematical assessment assumes a standard Byzantine environment where an attacker can control some nodes and manipulate message delivery timing unless they completely cut off network communication (Lamport et al., 1982). The main structural constraint for the network state is defined by the classical Byzantine limit:

$$n > 3f$$

where n is the total network voting weight (or total node count in an unweighted setting), and f is the maximum number of

arbitrary, malicious, or failed entities allowed in the system (Lamport et al., 1982). This approach allows the system to achieve non-blocking, livelock-free performance across fully asynchronous environments (Baird, 2016).

IV. IMPLEMENTATION

Hedera Hashgraph replaces traditional linear verification with two closely integrated mechanisms: the Gossip-about-Gossip protocol for sharing information and Virtual Voting for global transaction ordering (Baird, 2016).

A. Gossip-about-Gossip

Information spreads across the network using an optimized gossip protocol. When Node Alpha starts communication, it randomly picks Node Beta from the active group and sends its complete local view of the hashgraph DAG that Node Beta hasn't recorded yet. Once Node Beta receives this data, it creates a new event in its local memory (Baird et al., 2019). This new event acts as a cryptographic container. It holds a series of new client transactions and two structural references: the self-parent hash, pointing to Node Beta's last local event, and the other-parent hash, which points directly to the last event created by Node Alpha before the transmission. Because every event includes the hashes of its parents, nodes do not just share transaction data; they share a verifiable record of the network's communication history.

This is why the protocol is called Gossip-about-Gossip: nodes discuss the history of the gossip itself (Baird, 2016). The network load is optimized since nodes send only small cryptographic hashes for graph edges. This speeds up the growth of the DAG across all participating nodes without serialization limits..

B. Virtual Voting

The key feature of Hedera's consensus model is Virtual Voting. In standard Byzantine Fault Tolerant systems, nodes achieve consensus by sending a series of explicit vote messages over multiple rounds. This results in an $O(n^2)$ or $O(n^3)$ bandwidth bottleneck, which harms performance as the network grows (Castro & Liskov, 1999).

Hedera addresses this issue by calculating votes locally. Each validator node keeps an identical, cryptographically validated copy of the hashgraph DAG. This means each node can see exactly when and how every other node received a specific transaction (Baird, 2016).

Using the local DAG as a matrix, a node runs a deterministic voting algorithm to find out how any other node would vote in a traditional message-passing setup. Nodes vote virtually by running the same calculations over their local graphs, resulting in identical consensus outputs without transmitting any votes over the network (Baird et al., 2019).

The process for virtual voting follows specific structural definitions within the DAG:

Round Created: An event's round R is set based on its parents. A starting event is labeled Round 1. For any subsequent event e , its round starts as the highest round of its

parents, $R = \max(R_{\text{self_parent}}, R_{\text{other_parent}})$. However, if e can strongly see a supermajority of "witness" events from round R , its round goes up by one, making it $R + 1$ (Baird, 2016).

Witnesses: The first event created by a validator node in any round is called a witness. These events are crucial for voting and round advancement.

Seeing and Strongly Seeing: An event e_1 is said to see an older event e_2 if there is a valid, downward path from e_1 to e_2 within the DAG. Importantly, e_1 strongly sees e_2 if it can trace valid downward paths to e_2 through a set of intermediate events created by a distinct supermajority of nodes. This means the total voting weight of these intermediate creators exceeds $\frac{2}{3}$ of the total network weight ($> \frac{2}{3}n$) (Baird, 2016).

After confirming a witness event's round, the network enters the validation phase. Higher-round witnesses act like an internal jury, voting on whether lower-round witnesses are accepted into the permanent consensus history. If a round $R+1$ witness strongly sees a round R witness, it registers an internal vote of true (Baird, 2016). Since every node runs this identical state machine over its local graph, they all reach the same Byzantine agreement outcome independently. No network coordination or explicit voting phases are needed.

C. Consensus Timestamping and Ordering

Format and save your graphics using a suitable graphics processing program that will allow you to create the images as PostScript (PS), Encapsulated PostScript (.EPS), Tagged Image File Format (.TIFF), Portable Document Format (.PDF), JPEG, or Portable Network Graphics (.PNG). These programs can resize them and adjust the resolution settings. If you created your source files in one of the following programs you will be able to submit the graphics without converting to a PS, EPS, TIFF, PDF, or PNG file: Microsoft Word, Microsoft PowerPoint, or Microsoft Excel. Though it is not required, it is strongly recommended that these files be saved in PDF format rather than DOC, XLS, or PPT. Doing so will protect your figures from common font and arrow stroke issues that occur when working on the files across multiple platforms. When submitting your final files, your graphics should all be submitted individually in one of these formats along with the manuscript.

D. Sizing of Graphics

Once the network decides which witness events are accepted, it calculates the exact global position and order of every transaction within the DAG. For any transaction, its global order is defined by its consensus timestamp.

To find the consensus timestamp for an event e , the system identifies all the witness events in the first round that can see e . The local arrival times at each validator node that created those witnesses are gathered into an ordered set. The network takes the median of this set as the final, immutable consensus timestamp for event e (Baird, 2016).

$$\text{Consensus Timestamp} = \text{median}\{t_{\text{arrival}}^1, t_{\text{arrival}}^2, \dots, t_{\text{arrival}}^m\}$$

Sorting transactions chronologically by their consensus timestamps establishes a fair, global transaction sequence. If two transactions have the same median timestamp, the tie is resolved by comparing the digital signatures of the events that contain them.

This method ensures Fair Ordering: a malicious node cannot manipulate the order of transactions by prioritizing its own blocks or front-running user submissions. The global sequence is determined by the collective median arrival time across the entire network (Baird et al., 2019).

V. RESULTS AND DISCUSSION

The separation of transaction dissemination from global block production creates performance metrics that set Hedera Hashgraph apart from older linear distributed ledgers.

A. Throughput and Latency Profiling

Traditional linear blockchains face limits due to serial block confirmation, which forces a trade-off between throughput and latency. Bitcoin processes about 7 Transactions Per Second (TPS) with confirmation delays of minutes to hours (Nakamoto, 2008). Ethereum's base layer handles roughly 15 to 30 TPS.

In contrast, Hedera's DAG structure allows nodes to ingest and spread events in parallel, removing the wait for global blocks. The main bottleneck shifts from consensus protocol overhead to network bandwidth and local CPU processing limits (Baird, 2016).

Enterprise implementations of Hedera's consensus system consistently support over 10,000 TPS for token transfers and data integrity tasks. By using virtual voting instead of traditional multi-round vote messaging, transaction finality latencies drop to less than a second or low seconds (3 to 5 seconds globally), providing a certain outcome where traditional models only offer probabilities (Baird et al., 2019).

B. Comparative Analysis of Security Guarantees

Hedera's main security benefit is its mathematical basis in Asynchronous Byzantine Fault Tolerance (aBFT), the strongest security classification for distributed state machines (Lamport et al., 1982; Baird, 2016).

Many popular consensus protocols depend on partial synchrony models, assuming that messages will always arrive within a set time frame (Δ). If an attacker delays messages beyond this window through tactics like BGP hijacking or targeted firewalls, the system's assumptions break, risking double-spend attacks or halts (Vukolić, 2015).

Hedera's aBFT protocol works without relying on timing assumptions, making it robust against communication delays, network splits, and lost packets (Baird, 2016). As long as the total weight of honest validators remains above $\frac{2}{3}$, the network will continue to reach accurate consensus updates safely.

C. Architectural Limitations

While the advantages of the hashgraph framework are clear, there are important trade-offs and limitations:

Storage Overhead: Because nodes constantly gossip about gossip, the local Directed Acyclic Graph grows quickly over time. Keeping the historical graph structure requires significant storage, leading to the need for specialized state-pruning algorithms and mirror-node systems to manage historical records away from core consensus validators (Wang et al., 2020).

Bandwidth Intractability: The gossip protocol needs ongoing peer-to-peer data transfer. In networks with thousands of independent nodes, total bandwidth use can become a limiting factor. The system works best with strong, high-capacity network connections.

Decentralization Mechanics: Achieving true aBFT finality in a permissionless system requires strong sybil protection. If anyone can create millions of free virtual identities, they could easily bypass the $\frac{1}{3}$ fault tolerance requirement. Hedera addresses this by weighting votes based on economic stakes (Proof-of-Stake), shifting the security focus from identity constraints to economic limits (Baird et al., 2019).

VI. CONCLUSION AND FUTURE WORK

Hedera Hashgraph tackles the main flaws of traditional linear blockchains by replacing sequential block production with a Directed Acyclic Graph (DAG) and an asynchronous Byzantine Fault Tolerant (aBFT) consensus protocol. By merging a gossip-about-gossip method with virtual voting, Hedera removes the high costs of traditional consensus mechanisms, allowing for high transaction throughput, low latency, and fair transaction ordering without complex sharding or off-chain scaling layers (Baird, 2016).

Future research in this area should focus on:

- Designing improved state-pruning algorithms to lessen the long-term storage needs of rapidly growing DAG structures.
- Exploring how leaderless aBFT protocols perform in fragmented networks with variable bandwidth.
- Optimizing protocols for cross-chain interoperability to link DAG-based networks with traditional linear blockchain platforms without losing speed or security guarantees.

REFERENCES

- Baird, L. (2016). The swirlds hashgraph consensus algorithm: Fair, fast, byzantine fault tolerance. Swirlds Tech Report SWIRLDS-TR-2016-01.
- Baird, L., Harmon, M., & Madsen, P. (2019). Hedera: A public hashgraph network & governing council (Whitepaper). Hedera Hashgraph.
- Bano, S., Sonnino, A., Al-Bassam, M., Azouvi, S., McCorry, P., Meiklejohn, S., & Danezis, G. (2019). SoK: Consensus in the age of blockchains. Proceedings of the 1st ACM Conference on Advances in Financial Technologies (AFT '19), 183–198. <https://doi.org/10.1145/3318041.3355458>
- Buterin, V. (2017). The meaning of decentralization. Medium.
- Castro, M., & Liskov, B. (1999). Practical Byzantine fault tolerance. Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI '99), 173–186.
- DeCandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., Sivasubramanian, S., Voshall, P., & Vogels, W. (2007). Dynamo: Amazon's highly available key-value store. ACM SIGOPS Operating Systems Review, 41(6), 205-220.
- King, S., & Nadal, S. (2012). PPCoin: Peer-to-peer crypto-currency with proof-of-stake.
- Lamport, L., Shostak, R., & Pease, M. (1982). The Byzantine generals problem. ACM Transactions on Programming Languages and Systems (TOPLAS), 4(3), 382-401.

Nakamoto, S. (2008). Bitcoin: A peer-to-peer electronic cash system. Decentralized Business Review.

Vukolić, M. (2015). The quest for scalable blockchain fabric: Proof-of-work vs. BFT replication. International Workshop on Open Problems in Network Security (iNetSec 2015), 112-125. Springer.

Wang, Q., Li, R., Wang, Q., & Chen, S. (2020). Exploring the scalability of blockchain-based distributed ledgers: A comprehensive survey. IEEE Access, 8, 115002-115032.

Zheng, Z., Xie, S., Dai, H., Chen, X., & Wang, H. (2017). An overview of blockchain technology: Architecture, consensus, and future trends. 2017 IEEE International Congress on Big Data (BigData Congress), 557-564.

Mary Q. M. Mwangi (ICS4.1C, 158716).

Emmanuel C. Keter (ICS4.1C, 134976).

Louis M. Karanja (ICS4.1C, 169275).